COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 / Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	October 2, 2026	**DATE SUBMITTED:**	October 2, 2026
NAME:	CARDENAS, Lea Marie DE VERA, Sophia	**INSTRUCTOR:**	Engr. Jimlord Quejado

HANDS-ON ACTIVITY #10.1 GRAPHS

1. PROCEDURE OUTPUT

A.2 Adjacency Matrix

CODE

```
#include <iostream>
#include "GRAPH.H"
#include "ILOB1_DFS.H"
#include "ILOB2_BFS.H"

int main(){

    // A.2 Adjacency Matrix
    MatrixGraph graph;
// Undirected Graph
    graph.addVertex('A');
    graph.addVertex('B');
    graph.addVertex('C');
    graph.addVertex('D');
    graph.addVertex('E');

    graph.addEdge('A', 'C');
    graph.addEdge('A', 'D');
    graph.addEdge('A', 'B');
    graph.addEdge('B', 'E');
    graph.addEdge('C', 'D');
    graph.addEdge('D', 'E');

    std::cout << "A2 - Adjacency Matrix" << std::endl;
    graph.displayMatrix();
```

```
    MatrixGraph weightedGraph;
// Weighted Graph
    weightedGraph.addVertex('A');
    weightedGraph.addVertex('B');
    weightedGraph.addVertex('C');
    weightedGraph.addVertex('D');
    weightedGraph.addVertex('E');

    weightedGraph.addWeightedEdge('A', 'B', 4);
    weightedGraph.addWeightedEdge('A', 'C', 3);
    weightedGraph.addWeightedEdge('C', 'D', 2);
    weightedGraph.addWeightedEdge('D', 'E', 1);
    weightedGraph.addWeightedEdge('D', 'B', 1);
    weightedGraph.addWeightedEdge('E', 'A', 1);

    std::cout << "A2 - Weighted Adjacency Matrix" << std::endl;
    weightedGraph.displayWeightedMatrix();

    Graph g;
    g.addVertex('A');
    g.addVertex('B');
    g.addVertex('C');
    g.addVertex('D');
    g.addVertex('E');

    g.addEdge('A', 'B');
    g.addEdge('A', 'C');
    g.addEdge('A', 'D');
    g.addEdge('B', 'E');
    g.addEdge('C', 'D');
    g.addEdge('D', 'E');

    std::cout << std::endl;
    std::cout << "A3 - Adjacency List" << std::endl;
    std::cout << std::endl;
```

```
c:\Users\TIPQC\OneDrive\Documents\DSA De Vera_4-7pm\stack4.1\hoa>main.exe
A2 - Adjacency Matrix
   A B C D E
A  0 1 1 1 0
B  1 0 0 0 1
C  1 0 0 1 0
D  1 0 1 0 1
E  0 1 0 1 0
A2 - Weighted Adjacency Matrix
   A B C D E
A  0 4 3 0 0
B  0 0 0 0 0
C  0 0 0 2 0
D  0 1 0 0 1
E  1 0 0 0 0
```

OBSERVATION ANALYSIS

- *This is an Adjacency Matrix where it graphs the vertices given using the function MatrixGraph() and Graph() functions. In this case, the vertices A to E are first added to the graph, and addEdge() connects two vertices by placing 1 in both corresponding matrix positions because the graph is undirected. The displayMatrix() prints the array matrix, where 1 means the two vertices are connected and 0 means there's none.*

A.3 - Adjacency List

CODE

```
    // A.3 - Adjacency List

    DirectedGraph directedGraph;
//Undurected / Directed Graph
    directedGraph.addVertex('A');
    directedGraph.addVertex('B');
    directedGraph.addVertex('C');
    directedGraph.addVertex('D');
    directedGraph.addVertex('E');

    directedGraph.addEdge('A', 'B');
    directedGraph.addEdge('A', 'C');
    directedGraph.addEdge('B', 'E');
    directedGraph.addEdge('C', 'D');
    directedGraph.addEdge('D', 'B');
    directedGraph.addEdge('D', 'E');

    std::cout << std::endl;
    std::cout << "A3 - Directed Graph Adjacency List" << std::endl;
    std::cout << std::endl;

    directedGraph.display();
```

```
A3 - Adjacency List

A -> B -> C -> D -> NULL
B -> A -> E -> NULL
C -> A -> D -> NULL
D -> A -> C -> E -> NULL
E -> B -> D -> NULL


A3 - Directed Graph Adjacency List

A -> B -> C -> NULL
B -> E -> NULL
C -> D -> NULL
D -> B -> E -> NULL
E -> NULL
Graph:
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},
```

OBSERVATION ANALYSIS

- *This is an adjacency list (both directed and undirected). So basically, this type of listing uses adjList which makes the connections of each vertex easier to see by listing the vertices that are directly connected to it.*
- *For the directed adjacency list, the graph uses directions on which vertices it'll go. And that destination is directed/instructed by the addEdge() function.*
- *Its difference in the undirected adjacency list is that it has no specific direction to follow. The connection works both ways between the connected vertices.*

B.1 - Depth-First Search

CODE

```
// ILO B1
    //DFS File Code
    GraphList<unsigned> G = create_DFS_graph<unsigned>();

    std::cout << "Graph:\n";
    std::cout << G << std::endl;

    std::cout << "Recursive DFS Order of vertices: ";
    std::vector<size_t> order = recursive_DFS(G, 1);
    for (size_t v : order)
        std::cout << v << " ";

    std::cout << "\nIterative DFS Order of vertices: ";
    order = iterative_DFS(G, 1);
    for (size_t v : order)
        std::cout << v << " ";
```

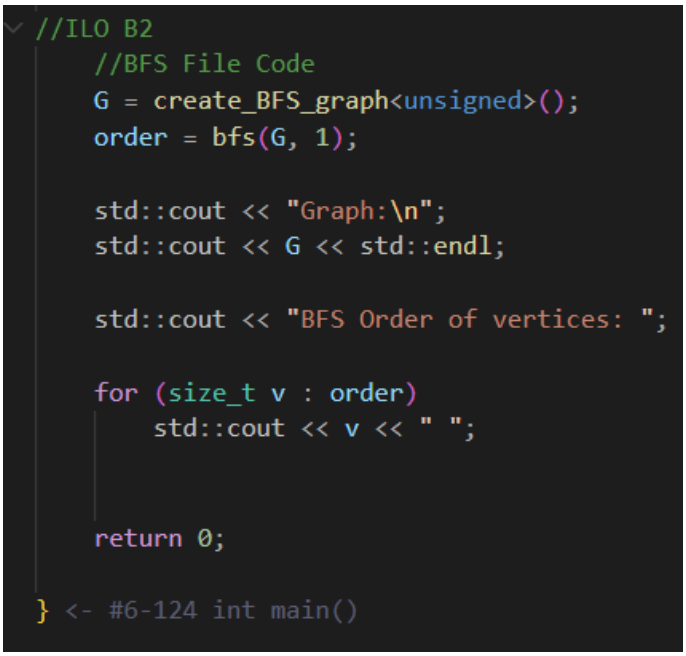
```
Recursive DFS Order of vertices: 1 2 5 4 3 7 6 8
Iterative DFS Order of vertices: 1 2 5 4 3 7 6 8 Graph:
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},
```

OBSERVATION ANALYSIS

- *This ILO B1 is a DFS implementation where it starts from any vertex. The dfs() function first marks the current vertex as visited using the visited set and adds it to the order vector. In this case, it was element 1. It starts to scan and store in the "order". Once checked, it is then placed in the visited list then checks every outgoing edge of that vertex. If the destination vertex has not been visited, the function calls itself recursively using that vertex.*
- *We thought this takes a long runtime before it finishes the whole list, especially when the data array/vector is a lot.*

\

B.2 - Breadth First Search

CODE	 
OBSERVATION ANALYSIS	- ***BFS implementation, as said from the earlier discussion, uses a queue. The starting vertex is inserted into the queue and marked as visited. The algorithm then repeatedly removes the vertex at the front of the queue. It scans and processes all the outgoing edges and adds every unvisited neighboring vertex.*** - ***We saw a video discussion on youtube and thought BFS is somehow more efficient and less-time consuming than DFS.***

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

- **DFS because the person goes as deep as possible from a location, backtracks when there are no unvisited neighbors, and it explores the next branch from the previous vertex.**

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

- **Tree pre-order and graph BFS both rely on a stack, level–order and graph BFS both rely on a queue. Their only difference is that a tree has no cycles that the traversal does not need to visit a set and the graph traversal can be visited a set.**

3. In the performed code, what data structure is used to implement the Breadth First Search?

- **It used a queue code to implement Breadth First Search. Because the queue stores the vertices waiting to be visited and it's a first in first out order that makes the search move level by level. And it also uses a set to track visited vertices and a vector to store the order.**

4. How many times can a node be visited in the BFS?

- **Node can be visited only once in BFS, because visited sets prevent vertex from being processed again even if it is added to the queue more than once. It also stops BFS from looping forever in graphs that contain cycles.**

Conclusion:

- **In conclusion, graph traversal algorithms are useful for systematically visiting vertices in a graph. Depth-First Search (DFS) is appropriate when the goal is to explore a path as deeply as possible before backtracking, while Breadth-First Search (BFS) explores vertices level by level. In the given BFS implementation, a queue is used to process vertices in FIFO order, and a visited set ensures that each vertex is processed only once. Understanding the difference between DFS and BFS helps determine which traversal algorithm is most suitable for a particular problem.**

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.